Uniwersytet Kazimierza Wielkiego | Wydział Informatyki

Inżynieria wiedzy i maszynowe uczenie · spotkanie 3 · kamień milowy A

System, który się tłumaczy

Wyjaśnienia, weryfikacja bazy wiedzy i interfejs

mgr inż. Marek Andryszczyk
Wydział Informatyki UKW · Informatyka, II rok · 2026/27

Cele dzisiejszego spotkania

1. **System wyjaśnia decyzje**
   „jak?”, „dlaczego?” i „co to?”

2. **Znajdujesz błędy w bazie wiedzy**
   najpierw na papierze, potem programem

3. **Budujesz interfejs w Streamlit**
   formularz tworzony z pliku JSON

4. **Zaliczasz kamień milowy A**
   aplikacja na Twojej bazie

Plan dzisiejszych zajęć: 90 minut

00–15	Teoria: wyjaśnienia, weryfikacja i walidacja, Streamlit	teoria
15–30	Moduł wyjaśnień: „jak?”, „dlaczego?”, „co to?” w silniku	kod
30–40	Błędy w bazie wiedzy na papierze · karta pracy, zad. 1	ćwiczenie
40–60	Weryfikator bazy: TODO 3–7 w weryfikacja.py	kod
60–80	Interfejs Streamlit na własnej bazie: TODO 1–4 w app.py	kod
80–90	Kamień milowy A: rozmowa przy stanowisku, zadanie Z3	organizacja

01

Wyjaśnienia

Jak doszedłeś do wniosku? Dlaczego o to pytasz? · ok. 6 min

Dlaczego system musi się tłumaczyć?

Użytkownik

ufa radzie, którą rozumie

Ekspert

sprawdza tok rozumowania bez czytania kodu

Autor bazy

ślad wskazuje regułę, która dała dziwny wynik

Przepisy

przejrzystość systemów AI, np. AI Act

Przewaga systemu regułowego

Ślad odpalonych reguł jest dokładnym zapisem rozumowania, a nie opisem dopisanym po fakcie. Model językowy na spotkaniu 7 będzie tylko redagował tekst na podstawie takiego śladu.

Dwa pytania użytkownika: „jak?” i „dlaczego?”

PO KONSULTACJI

„Jak doszedłeś do wniosku?”

- ślad: reguły, które odpaliły, po kolei
- każda reguła z metaforą
- na końcu opis „co to?”

W TRAKCIE KONSULTACJI

„Dlaczego o to pytasz?”

- reguła, którą system właśnie sprawdza
- parametr powod z wnioskowania wstecz
- użytkownik wpisuje ? zamiast odpowiedzi

`metafora:` dlaczego reguła ma sens, językiem laika

`coto:` co to jest? opis decyzji końcowej

Wyjaśnienie „jak?” dla espresso

```
wyjasnij_jak(baza, slad) dla 18°C, rano, brak, kofeina tak
1. R2: JEŚLI temperatura <= 22 TO rodzaj_kawy = 'gorąca'
   → Jest chłodno (do 22°C), więc proponuję kawę na gorąco.
2. R4: JEŚLI kofeina_ok == 'tak' ORAZ pora_dnia == 'rano' TO moc = 'mocna'
   → Rano przyda się mocniejsza dawka kofeiny.
3. R15: JEŚLI rodzaj_kawy == 'gorąca' ORAZ dodatek == 'brak' ORAZ moc == 'mocna'
          TO kawa = 'espresso'
   → Kawa na gorąco, bez dodatków, mocna.
```

Co to jest? Mała, mocna kawa z ekspresu ciśnieniowego.

Ślad: `R2 → R4 → R15`

Wyjaśnienie „dlaczego?” w trakcie konsultacji

```
konsola: wnioskowanie wstecz z obsługą „?”
Jaka jest temperatura na zewnątrz (°C)? (? = dlaczego pytam) 18
Czy możesz pić kofeinę? (? = dlaczego pytam) tak
Jaka jest pora dnia? (? = dlaczego pytam) ?
   Pytam, bo sprawdzam regułę: R4: JEŚLI kofeina_ok == 'tak'
   ORAZ pora_dnia == 'rano' TO moc = 'mocna'
Jaka jest pora dnia? (? = dlaczego pytam) rano
```

Pytanie	Powód
temperatura	R1
kofeina_ok	R3
pora_dnia	R4
dodatek	R12

powod to reguła, która potrzebuje danej. Przekazujesz ją już w rekurencji wstecz (powod=regula) — teraz ją wykorzystujesz, gdy użytkownik wpisze **?**.

Moduł wyjaśnień w silniku

se/silnik.py: dopisz

```
def opisz_regule(regula):            # gotowe
    warunki = " ORAZ ".join(f"{a} {op} {w!r}"
                            for a, op, w in regula["jesli"])
    atrybut, wartosc = regula["to"]
    return f"{regula['id']}: JEŚLI {warunki} TO {atrybut} = {wartosc!r}"

def wyjasnij_jak(baza, slad):        # napisz sam
    po_id = {r["id"]: r for r in baza["reguly"]}
    kroki = []
    for nr, id_reguly in enumerate(slad, start=1):
        …  # numer i opis reguły, potem „   → metafora”
    return kroki

def coto(baza, slad):                # gotowe
    …  # pole coto ostatniej reguły śladu, która je ma
```

Obsługa „?”

W zapytaj_konsola: gdy odpowiedź to ?, wypisz opisz_regule(powod) i zapytaj jeszcze raz.

Sprawdź

python se/silnik.py → odpowiedz ? na pytanie o porę dnia. Jaką regułę pokazał system i dlaczego tę?

02

Weryfikacja bazy wiedzy

Odpowiednik programu Cake z pakietu SPHINX · ok. 5 min teorii

Uniwersytet Kazimierza Wielkiego
Wydział Informatyki

Weryfikacja a walidacja

WERYFIKACJA

Czy baza jest zbudowana poprawnie?

- sprawdza program
- składnia, deklaracje, spójność reguł
- sprzeczności, duplikaty, pochłanianie

WALIDACJA

Czy baza zawiera właściwą wiedzę?

- sprawdza ekspert albo dane
- decyzje na nowych, prawdziwych przypadkach
- etykiety muszą być niezależne od silnika

Na spotkaniach 4–6: dane wygenerowane z Twojego silnika mierzą, czy drzewo odtwarza reguły. Nie dowodzą, że reguły są trafne.

Typowe błędy w bazie wiedzy

Problem	Przykład	Poziom
Sprzeczność	te same warunki: zimna ORAZ brak → cold brew i → espresso	BŁĄD
Wartość spoza dziedziny	dodatek == "syrop", a syropu nie ma na liście wartości	BŁĄD
Warunek niespełnialny	rodzaj_kawy == "letnia": żadna reguła tego nie ustala	BŁĄD
Cel nieosiągalny	żadna reguła nie ma wniosku o atrybucie celu	BŁĄD
Duplikat	dwie identyczne reguły (np. R9 i R17)	UWAGA
Pochłanianie	zimna ORAZ mleko ORAZ rano → iced latte, choć jest zimna ORAZ mleko → iced latte	UWAGA
Element nieużywany	atrybut wiek albo wartość wieczór nie występuje w żadnej regule	UWAGA

Pochłanianie i sprzeczność: ciche błędy



`warunki(A) ⊂ warunki(B)` i ten sam wniosek → B jest pochłaniana przez A: gdy B jest spełniona, A też.

Sprzeczność

```
R11: zimna ∧ brak → cold brew
R18: zimna ∧ brak → espresso
```

Dlaczego groźna?

Program działa bez komunikatu o błędzie: wygrywa pierwsza reguła, a druga po cichu nigdy nie odpala. Wykryje to tylko automatyczna weryfikacja.

Znajdź 10 problemów bez komputera

Karta pracy, zad. 1 · indywidualnie · 10 min

```
kawa_z_bledami.json (fragment; pozostałe: R1–R5, R7–R16 z kawa.json, R6 usunięta)

Atrybuty: dodatek ∈ {mleko, czekolada, brak}; rodzaj_kawy ∈ {zimna, gorąca} (wniosek);
          pora_dnia ∈ {rano, popołudnie, wieczór}; wiek (liczba)

R9:  JEŚLI rodzaj_kawy == zimna ORAZ dodatek == czekolada   TO kawa = chocolate cold coffee
R10: JEŚLI rodzaj_kawy == zimna ORAZ dodatek == mleko       TO kawa = iced latte
R11: JEŚLI rodzaj_kawy == zimna ORAZ dodatek == brak        TO kawa = cold brew
R17: JEŚLI rodzaj_kawy == zimna ORAZ dodatek == czekolada   TO kawa = chocolate cold coffee
R18: JEŚLI rodzaj_kawy == zimna ORAZ dodatek == brak        TO kawa = espresso
R19: JEŚLI rodzaj_kawy == zimna ORAZ dodatek == mleko ORAZ pora_dnia == rano TO kawa = iced latte
R20: JEŚLI rodzaj_kawy == letnia                            TO kawa = americano
R21: JEŚLI cukier == tak ORAZ dodatek == mleko              TO kawa = latte
R4:  JEŚLI dodatek == syrop                                 TO kawa = latte
```

weryfikuj(baza): pięć testów do napisania

TODO	Test	Poziom	Wskazówka
3	wartość spoza dziedziny	BŁĄD	tylko atrybuty z listą wartosci i operatory == oraz !=
4	nieużywany atrybut wejściowy / wartość	UWAGA	zbiory uzyte_atrybuty i uzyte_wartosci (pary)
5	warunek niemożliwy do spełnienia	BŁĄD	wytwarzane = {tuple(r["to"]) for r in reguly}
6	duplikat / sprzeczność / pochłanianie	U / B / U	pary reguł; warunki jako zbiory krotek; wa < wb
7	cel nieosiągalny	BŁĄD	any(r["to"][0] == baza["cel"] for r in reguly)

```
python se/weryfikacja.py wiedza/kawa_z_bledami.json
```

Raport weryfikacji: błędy i uwagi

```
python se/weryfikacja.py wiedza/kawa_z_bledami.json
[BŁĄD]  Powtórzony identyfikator reguły R4
[BŁĄD]  R20: wartość 'letnia' spoza dziedziny 'rodzaj_kawy'
[BŁĄD]  R20: warunek rodzaj_kawy == 'letnia' nigdy niespełniony
[BŁĄD]  R21: niezadeklarowany atrybut 'cukier'
[BŁĄD]  R4: wartość 'syrop' spoza dziedziny 'dodatek'
[UWAGA] Wartość 'wieczór' atrybutu 'pora_dnia' nieużywana
[UWAGA] Atrybut 'wiek' nie jest używany w żadnej regule
[UWAGA] R9 i R17 są identyczne (duplikat)
[UWAGA] R19 jest pochłaniana przez R10 (zbędna)
[BŁĄD]  R11 i R18 są sprzeczne: te same warunki
```

kawa_z_bledami.json

10 problemów

kawa.json

0 problemów

Twoja baza

BŁĘDY usuń, UWAGI uzasadnij w README



Czego raport nie gwarantuje

OGRANICZENIA WERYFIKATORA

- nakładające się przedziały liczbowe (> 20 i > 22)
- pokrycie wszystkich kombinacji wejść
- cykl w grafie wniosków (rozszerzenie)
- trafność wiedzy: to walidacja, nie weryfikacja
- 0 uwag ≠ baza kompletna

MINIMALNE TESTY PRZYPADKÓW

- każda decyzja końcowa co najmniej raz
- granica progu: 22 i 22,1°C
- brak wyniku: jawny komunikat
- dwie konkurujące reguły
- te same wejścia w obu trybach

03

Interfejs w Streamlit

Aplikacja webowa w czystym Pythonie · ok. 4 min teorii

Uniwersytet Kazimierza Wielkiego | Wydział Informatyki

Dlaczego Streamlit?

Czysty Python

bez HTML i JavaScript; kilkadziesiąt linii

Formularz z JSON

liczba → suwak, wybór → przyciski; działa dla każdej bazy

Czat na spotkaniu 7

ta sama aplikacja dostanie st.chat_input

terminal VS Code, katalog główny repozytorium

```
python -m streamlit run app.py

  Local URL: http://localhost:8501
```

Tkinter w IWiSE?

Streamlit to mniej kodu i czat w tej samej aplikacji.



Streamlit wykonuje skrypt od nowa

Interakcja
suwak, przycisk

→

Skrypt od góry
cały app.py jeszcze raz

→

st.form
obliczenia dopiero po „Doradź”

→

Wynik
st.success, st.expander

Stan między uruchomieniami

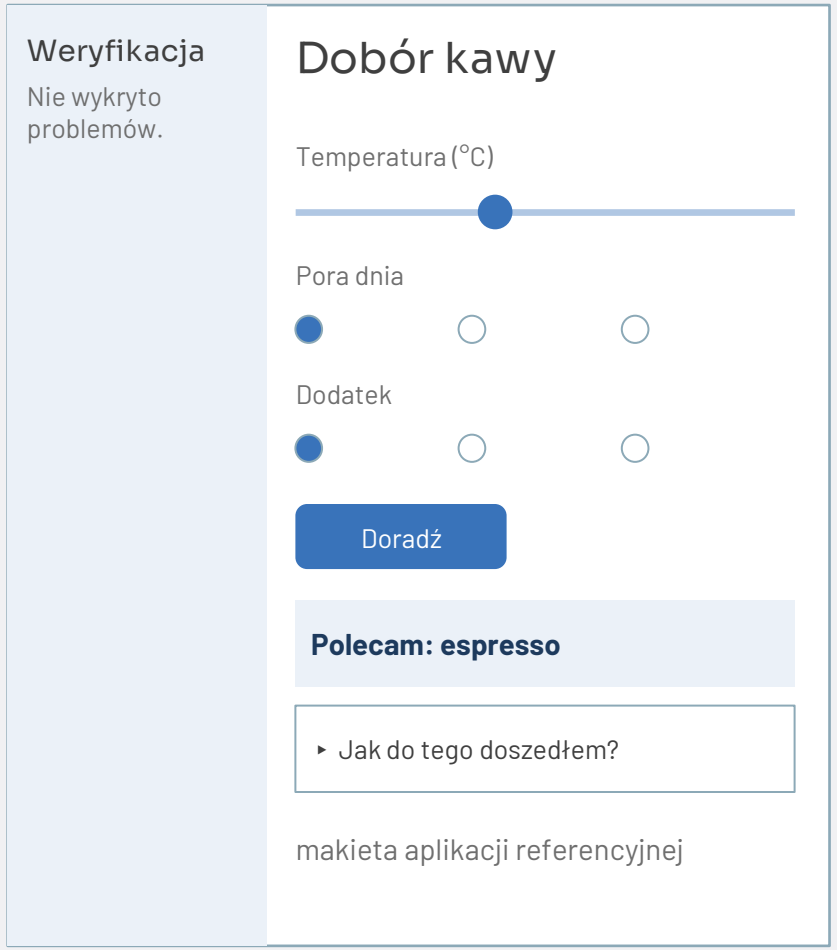
Zmienne Pythona znikają przy każdym uruchomieniu. Co ma przetrwać (np. historia rozmowy w czacie na spotkaniu 7), trzymamy w st.session_state.

Formularz budowany z pliku JSON

app.py

```
with st.form("konsultacja"):
    fakty = {}
    for nazwa, opis in baza["atrybuty"].items():
        if opis["typ"] == "liczba":
            lo, hi = opis["zakres"]
            fakty[nazwa] = st.slider(opis["pytanie"], lo, hi)
        elif opis["typ"] == "wybor":   # TODO 2
            fakty[nazwa] = st.radio(opis["pytanie"],
                                    opis["wartosci"], horizontal=True)
    wyslano = st.form_submit_button("Doradź")

if wyslano:
    wynik, slad = wnioskuj_w_przod(baza, fakty)
    …  # TODO 3: st.success / st.error, st.info(coto)
    …  # TODO 4: st.expander("Jak do tego doszedłem?")
```



makieta aplikacji referencyjnej

Kamień milowy A i zadanie Z3

Zadanie Z3 = kamień milowy A · termin: spotkanie 4 · 10 pkt

Rozmowa przy stanowisku (1–2 min)

- ✓ aplikacja Streamlit działa na Twojej bazie
- ✓ wynik + wyjaśnienie „jak?" + opis „co to?"
- ✓ raport bez błędów; uwagi uzasadnione w README
- ✓ odpowiadasz: „pokaż regułę, która dała ten wynik"

Kryterium Z3	pkt
moduł wyjaśnień: jak?, dlaczego?, co to?	3
weryfikator: 10/10, 0 fałszywych alarmów	3
własna baza: 0 błędów, uwagi i ograniczenia w README	2
aplikacja Streamlit na własnej bazie	2
Razem	10

Przed spotkaniem 4: silnik musi działać. Na spotkaniu 4 generuje etykiety danych do uczenia drzewa decyzyjnego.

Co zabierasz z dzisiejszych zajęć

01 Ślad reguł odtwarza rozumowanie: „jak?” po konsultacji, „dlaczego?” w jej trakcie.

02 Weryfikacja sprawdza budowę bazy, walidacja jej trafność.

03 Sprzeczność jest najgroźniejsza, bo działa po cichu.

04 Streamlit buduje formularz z pliku JSON; st.form chroni przed ciągłym przeliczaniem.

SPOTKANIE 4 · UCZENIE MASZYNOWE

Twój silnik wygeneruje dane, a drzewo decyzyjne nauczy się z nich reguł. Porównamy reguły drzewa z regułami eksperta.

Sprawdź się: czy potrafisz odpowiedzieć?

1. Czym różni się wyjaśnienie „jak?” od „dlaczego?”? Kiedy użytkownik zadaje każde z nich?
2. Wymień i scharakteryzuj pięć typów błędów w bazie wiedzy.
3. Czym różni się weryfikacja od walidacji bazy wiedzy?
4. Na czym polega pochłanianie reguł? Podaj przykład.
5. Dlaczego sprzeczne reguły są groźne, choć program działa bez błędów?

Bibliografia

[1] A. Wakulicz-Deja, A. Nowak-Brzezińska, M. Przybyła-Kasperek, R. Simiński, Systemy ekspertowe. Warszawa: EXIT, 2018 (rozdz. o weryfikacji baz wiedzy i module objaśniającym).

[2] AITECH, Pakiet SPHINX 4.0 (PC-Shell, Cake): dokumentacja programu.

[3] Streamlit, „Streamlit documentation: forms, session state". [Online]. Dostępne: https://docs.streamlit.io

[4] Python Software Foundation, „Set types: set, frozenset". [Online]. Dostępne: https://docs.python.org/3/library/stdtypes.html#set-types-set-frozenset

[5] Rozporządzenie (UE) 2024/1689 (AI Act): obowiązki przejrzystości systemów AI.

[6] Materiały kursu IWiMU 2026/27: se/silnik.py, se/weryfikacja.py, app.py, wiedza/kawa_z_bledami.json.

Uniwersytet Kazimierza Wielkiego | Wydział Informatyki

Dziękuję za uwagę



mgr inż. Marek Andryszczyk

Asystent badawczo-dydaktyczny
Wydział Informatyki UKW

marek.andryszczyk@ukw.edu.pl
ORCID: 0000-0003-2732-8631
andryszczyk.eu